

Chapter4:SQL

Database System Concepts

University of Guilan

Yasaman Boreshban

Fall 2025

Introduction to SQL and Database Implementation

- To perform logical design in a database management system, we need a database language.
- SQL (Structured Query Language) is the standard language used for relational database operations (from an application perspective).
- SQL Commands:
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
 - Data Control Language (DCL)
- Examples of DDL commands:
 - CREATE TABLE – Create a new table
 - DROP TABLE – Delete an existing table
 - ALTER TABLE – Modify the structure of a table
- In SQL statements, text values are enclosed in single quotes (' ').
- Some database systems also accept double quotes (" "), but single quotes are standard.

Defining and Dropping Databases and Schemas

- Creating a Database
 - **CREATE DATABASE** *DatabaseName*;
- Dropping a Database
 - **DROP DATABASE** *DatabaseName*;
- In most DBMSs, it is possible to define multiple schemas within a single database.
- Creating and Dropping a Schema
 - **CREATE SCHEMA** *SchemaName*;
 - **DROP SCHEMA** *SchemaName*;
- A database schema represents the logical design or structure of the database.
- It defines how data is organized and includes a collection of objects such as tables, views, and related elements.

Defining Tables with CREATE TABLE Command

```
CREATE TABLE TableName
    { ( columnName dataType [NOT NULL | UNIQUE]
      [DEFAULT defaultOption][CHECK (searchCondition)] [, ...] )}
    [PRIMARY KEY (listOfColumns), ]
    {[UNIQUE (listOfColumns),][, ...]}
    {[FOREIGN KEY (listOfForeignKeyColumns)
REFERENCES ParentTableName [(listOfCandidateKeyColumns)],
      [ON UPDATE referentialAction]
      [ON DELETE referentialAction]][, ...]}
    {[CHECK (searchCondition)][, ...]}
```

- You can also create temporary tables using:
- CREATE TEMPORARY TABLE *TableName* (...);
- A temporary table is deleted automatically when the SQL session ends.

Data Types in SQL

- Common SQL Data Types:
- Character Types
 - CHAR(n), VARCHAR(n)
- Bit String Types
 - BIT(n), BIT VARYING(n)
- Numeric Types
 - NUMERIC(p, q), REAL, INTEGER, SMALLINT, FLOAT(p), DOUBLE PRECISION
- Date and Time Types
 - DATE, TIME, TIMESTAMP, INTERVAL, ...
- Some DBMSs support special data types that allow advanced data storage, retrieval, and processing features. For example, PostgreSQL supports geographic data types.

Column Constraints in SQL

- Column Constraints specify rules that limit the type of data that can be stored in a column.
- **DEFAULT**: Specifies a default value for a column.
- **NOT NULL**: Ensures that a column cannot have NULL values.
- **UNIQUE**: Ensures that all values in a column (or a group of columns) are unique.
- **PRIMARY KEY**: Defines one or more columns as the primary key (unique and not null).
- **FOREIGN KEY ... REFERENCES ...** : Defines a foreign key that refers to a column (or columns) in another table.
- **CHECK**: Defines a condition that each value in the column must satisfy.

Example: Defining Tables in SQL

```
CREATE TABLE STT  
( STID          CHAR(8) NOT NULL,  
STNAME        CHAR(25),  
STLEV         CHAR(12),  
STMJR         CHAR(20),  
STDEID        CHAR(4)  
)  
PRIMARY KEY (STID);  
CHECK STMJR IN { 'bs', 'ms', 'doc', '???' };
```

```
CREATE TABLE COT  
( COID          CHAR(6) NOT NULL,  
COTITLE        CHAR(16),  
CREDIT         SMALLINT,  
COTYPE         CHAR(1),  
CODEID         CHAR(4),  
)  
PRIMARY KEY (COID);
```



constraint

Defining Tables in SQL

constraint



```
CREATE TABLE SCT
( STID          CHAR(8) NOT NULL ,
  COID          CHAR(6) NOT NULL ,
  TR           CHAR(1) ,
  YR           CHAR(5) ,
  GRADE        DECIMAL(2 , 2)
)
PRIMARY KEY ( STID, COID )
CHECK  0 ≤ GRADE ≤ 20
FOREIGN KEY (STID) REFERENCES STT (STID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
FOREIGN KEY (COID) REFERENCES COT (COID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
```


Dropping Tables in SQL

- DROP TABLE Command
- DROP TABLE tableName [CASCADE | RESTRICT];
- CASCADE:
 - Automatically removes all objects dependent on the table
 - (such as views or constraints like foreign keys).
- RESTRICT:
 - Prevents the table from being dropped if there are dependent objects.
 - By default, most systems use RESTRICT.
- Example:
 - DROP TABLE SCT;

Modifying Tables with ALTER TABLE

- ALTER TABLE Command

ALTER TABLE *tableName*

[ADD [COLUMN] *columnName* *dataType* [NOT NULL] [UNIQUE]

[DEFAULT *defaultOption*] [CHECK (*searchCondition*)]]

[DROP [COLUMN] *columnName* [RESTRICT | CASCADE]]

[ADD [CONSTRAINT [*constraintName*]] *tableConstraintDefinition*

[DROP [CONSTRAINT *constraintName* [RESTRICT | CASCADE]]

[ALTER [COLUMN] SET DEFAULT *defaultOption*]

[ALTER [COLUMN] DROP DEFAULT]

- Example: Add a new column "STATE" to the Student table

ALTER TABLE STT

ADD COLUMN STATE CHAR(10)

Database Schema and System Catalog

- In the database schema, we have three main types of commands:
 - Data Definition (DD)
 - Data Manipulation (DM)
 - Data Controller (DC)
- Advantages:
 - The schema describes how data is organized and stored.
 - The system stores all data in a set of tables.
- System Catalog (Data Dictionary):
 - Contains metadata (data about data).
 - Includes definitions, constraints, and user information.
 - Helps the DBMS manage and control data efficiently.

Table Data Base Language (TDBL)

- Operations in TDB: DML (Data Manipulation Language) Commands
 - SELECT → Retrieve data
 - INSERT → Add new data
 - DELETE → Remove existing data
 - UPDATE → Modify or refresh data

Retrieving Data with SELECT

```
SELECT [ALL | DISTINCT] item(s) list  
      FROM table(s) expression  
      [WHERE condition(s)]  
      [ORDER BY Col(s)]  
      [GROUP BY Col(s)]  
      [HAVING condition(s)]
```

- The output of a SELECT statement is always a table.
- DISTINCT is used to eliminate duplicate rows from the result set.
- The WHERE clause may include comparison operators such as: =, <>, >, <, >=, <=, BETWEEN, LIKE, IN

Examples of Data Retrieval

```
SELECT STT.STID AS SN,  
       STT.STNAME AS SName  
FROM   STT  
WHERE  STT.STMJR='phys'  
       AND  
       STT.STLEV='bs'
```

```
SELECT STT1.STID AS SN,  
       STT1.STNAME AS SName  
FROM   STT AS STT1  
WHERE  STT1.STMJR='phys'  
       AND  
       STT1.STLEV='bs'
```

Data Retrieval (Continued)

- Creating an alias for a subquery and sorting results

```
(SELECT S.*  
      FROM S) AS MyS  
ORDER BY SNAME or 2
```

- By default, sorting is ascending (ASC).
- To sort in descending order, use DESC.

Data Retrieval (Continued)

- Advanced features (pattern matching)

SELECT S#, CITY

FROM S

WHERE SNAME

$\left\{ \begin{array}{l} \text{LIKE} \\ \text{NOT LIKE} \end{array} \right\}$

$\left\{ \begin{array}{l} \%N' \\ 'M\%' \\ '_A_'\end{array} \right\}$



Ends with N



Starts with M



Exactly 5 characters, third is A

Data Retrieval (BETWEEN Operator)

- Returns the part numbers (P#) where the weight is between 5 and 15.

```
SELECT P#
```

```
FROM P
```

```
WHERE WEIGHT BETWEEN (5,15)
```

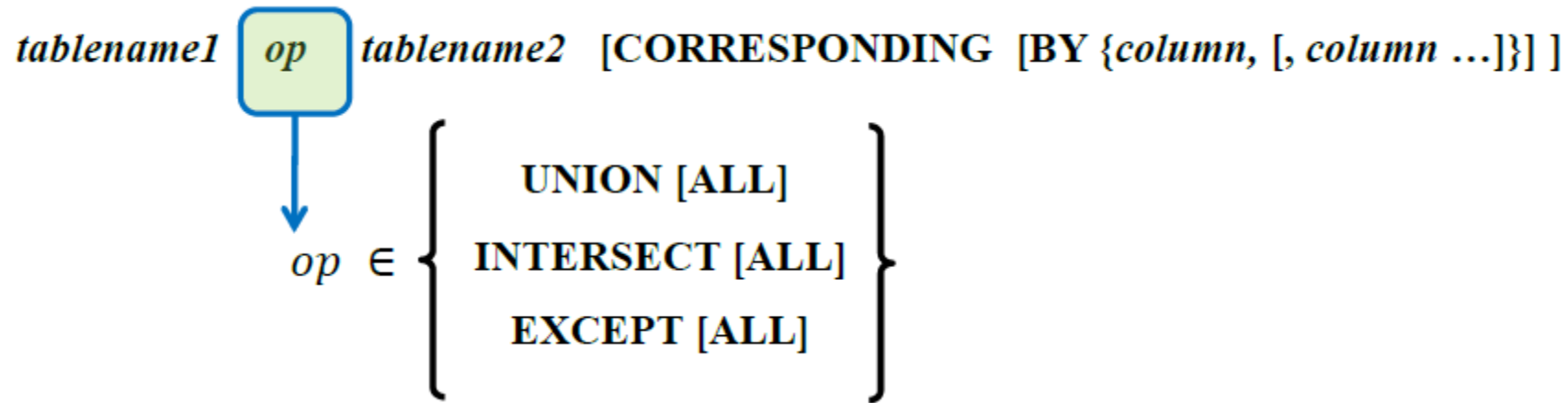
```
↳
```

```
WHERE WEIGHT >=5 AND WEIGHT <=15
```

Data Retrieval (Null)

```
SELECT S#, CITY
FROM S
WHERE STATUS { IS NULL
               IS NOT NULL }
```

Set Operations in SQL



- CORRESPONDING BY: Applies the operation only to the specified columns.
- CORRESPONDING: Applies the operation to all columns with the same names in both tables.
- Without CORRESPONDING: The operation applies to all columns of both tables.
- Columns (Headings) must have the same names and data types in both tables.
- ALL retains duplicate rows; otherwise, duplicates are removed automatically.

Examples of Set Operations

- Example : Suppliers number who produce at least one part

```
SELECT SP.S#,  
FROM SP
```

Example 2: Testing data consistency

Each person who produced a part must appear in the supplier table

```
SELECT SP.S#,  
FROM SP  
EXCEPT  
SELECT S.S#,  
FROM S
```

Examples of Set Operations

- Example: Find suppliers who do not produce any parts

```
SELECT S.S#,  
      FROM S  
      EXCEPT  
SELECT SP.S#,  
      FROM SP
```

- Returns supplier numbers (S#) that are in table S but not in table SP.
- In other words, suppliers who have not produced any parts.

Aggregate (Group) Functions

- Aggregation Functions:
 - AVG() → Average value
 - MIN() → Minimum value
 - MAX() → Maximum value
 - SUM() → Total sum
 - COUNT(*) / COUNT(column) → Counts rows or non-null values

Example of Aggregate Functions:

- Find the maximum status of suppliers in cities c1 or c2.

```
SELECT MAX ( STATUS ) AS SMAX  
FROM S  
WHERE CITY='c1'  
OR  
CITY='c2'
```

Example of Aggregate Functions

- Example 1: Count part types produced by suppliers

```
SELECT COUNT (DISTINCT P#) AS N1  
FROM SP
```

- Example 2: Count all types of parts

```
SELECT COUNT (*) AS N2  
FROM P
```


Example of Aggregate Functions

- Example : Find total quantity of parts produced by supplier S2

```
SELECT SUM(QTY) AS N3
FROM SP
WHERE S# = 's2'
```

GROUP BY Clause

- The GROUP BY clause divides the rows returned by the FROM clause into groups having the same values in one or more columns.
- Example:
- Find the total number of parts supplied by each supplier.

```
SELECT S# AS SN, SUM (QTY) AS SQ  
FROM SP  
GROUP BY S#
```

Example of GROUP BY

SN	SQ
s1	280
s2	100
s3	203
...	...



S#	P#	QTY
s1	p1	...
s1	p2	...
s1	p4	...
s2	p2	...
s2	p3	...
s3	p5	...
...	...	...

HAVING Clause

- The HAVING clause is used to specify conditions on groups (not on individual rows).
- Similar to WHERE, but applied after grouping.
- Example:
- Find supplier numbers who have supplied more than 100 parts.

```
SELECT S#  
  
FROM SP  
  
GROUP BY S#  
  
HAVING SUM(QTY) > 100
```

Practice: GROUP BY and HAVING

- List the student numbers who have taken more than 20 credits in the second semester of the academic year 1387–1388.

```
SELECT STID
FROM ENROLL
WHERE TERM = 2
      AND YEAR = '87-88'
GROUP BY STID
HAVING SUM(CREDITS) > 20;
```

Practice: GROUP BY and HAVING

- List the student numbers who have taken more than 7 courses in the second semester of the academic year 1387–1388.

```
SELECT STID
FROM ENROLL
WHERE TERM = 2
      AND YEAR = '87-88'
GROUP BY STID
HAVING COUNT(COURSE_ID) > 7;
```

Data Manipulation: INSERT, Update, Delete

- Insert:

```
INSERT INTO table-name [(col1,col2, ...)]  
VALUES ( one row ) | subquery
```

- Delete:

```
DELETE FROM table-name  
WHERE condition(s) / subquery
```

Example: Row Insertion (Complete and Partial Row)

```
INSERT INTO STT
VALUES ( '222' , 'st2' , 'IT' , 'bs' , 'D17' )

INSERT INTO STT
VALUES ( '333' , 'st3' , Null , 'ms' , Null )
```


Example - Group Insertion

- Create a temporary table and Insert data of M.Sc. Computer students into T1:

```
CREATE TEMPORARY TABLE T1
    ( STN, .... )

INSERT INTO T1
    ( SELECT STT.*
      FROM STT
      WHERE STJ = 'comp'
        AND
          STL = 'ms' )
```

Example- Delete:

- Delete the course 'com111' for student 88104444.

```
DELETE FROM STOCOT  
WHERE STID = 88104444  
AND  
COID = 'COM111'
```

- Should this deletion be propagated to other tables?

Example: DELETE from Multiple Tables

```
DELETE FROM DEPT  
WHERE DEID = 'D333'
```

```
UPDATE STT  
SET DEID = 'Null'  
WHERE DEID = 'D333'
```

Data Manipulation: Update

- Update

UPDATE *table-name*

SET *col = value / expression [, col = value / expression] ...*

⋮

WHERE *condition(s) / subquery*

Example: Updating Multiple Rows

- Example:
- Set the credit of all practical courses of the teaching group D11 to 1.

```
UPDATE  COT
SET    CREDIT = '1'
WHERE  COTYPE = 'p' AND  CODEID = 'D11'
```

Example: Updating Across Multiple Table

- Change student ID 88107777 to 88104444 in both tables STT and STCOT.

```
UPDATE  STT
      SET  STID = 88104444
      WHERE STID = 88107777
```

```
UPDATE  STCOT
      SET  STID = 88104444
      WHERE STID = 88107777
```

- What happens if the second UPDATE fails?

Practice: Update

- Example:
- Set the grade of students in teaching group D111 for the course 'com222' in the 2nd semester of year 85–86 as 'Incomplete'.

```
UPDATE STCOT
SET STCOT.GRADE = 'U'
WHERE STCOT.TR = '2' AND STCOT.YRYR = '85-86'
AND STCOT.COID = 'COM222'
AND STID IN (SELECT STID
              FROM STT
              WHERE STT.STDEID = 'D111');
```

Retrieving Data from More Than One Table

- Example:
- Show the names of suppliers who supply part 'p2'.

```
SELECT SNAME  
FROM S, SP  
WHERE SP.S# = S.S# AND SP.P# = 'p2'
```

- Equivalent to Join operation in relational algebra
- Cross product in SQL:

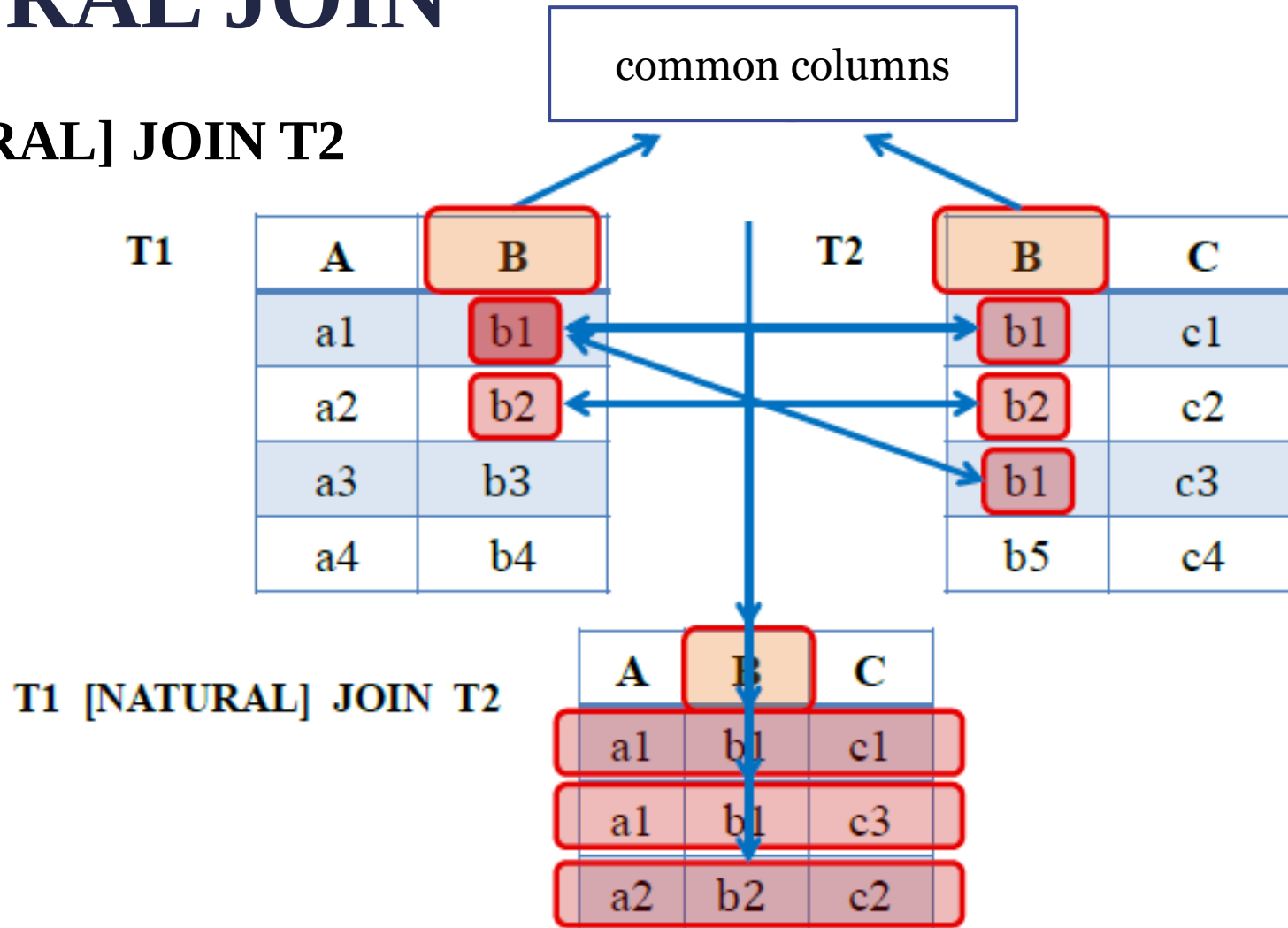
```
SELECT T1.*, T2.*  
FROM T1, T2
```


JOIN Operator

- NATURAL JOIN (Non-mathematical presentation)
- T1 [NATURAL] JOIN T2
- Combines rows from both tables where the values in all common columns (same name and type) are equal.
- Eliminates duplicate columns from the result.
- Example:
 - T1(A, B)
 - T2(B, C)
- Result:
 - T1 [NATURAL] JOIN T2 $\rightarrow$ (A, B, C)

NATURAL JOIN

T1 [NATURAL] JOIN T2



JOIN Operator

- Concept of JOIN:
- A JOIN combines rows from two tables whenever the values of one or more common columns are equal.
- Example:
 - Show the names of suppliers who supply part 'p2'.

```
SELECT SNAME
```

```
FROM S [NATURAL] JOIN SP
```

```
WHERE P# = 'p2'
```

Example:

S

S#	SNAME	...
s1	sn1	...
s2	sn2	...
s3	sn3	...
s3	sn4	...
...	...	...

SP

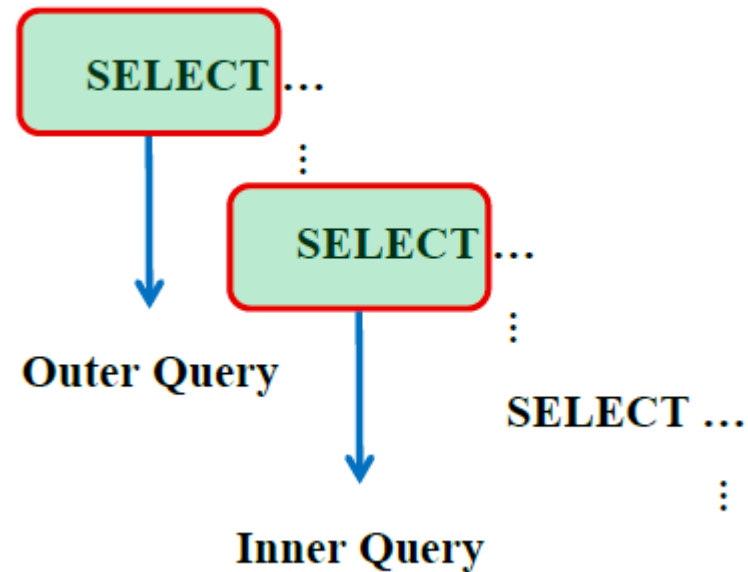
S#	P#	QTY
s1	p1	100
s1	p2	120
s1	p3	500
s2	p1	50
...	...	...

S [NATURAL] JOIN SP

S#	SNAME	...	P#	QTY
s1	sn1	...	p1	100
s1	sn1	...	p2	120
s1	sn1	...	p3	500
s2	sn2	...	p1	50
...	...	...	...	...

SubQuery

- Subquery – Nested SELECT Statement



SubQuery– Membership Operator (IN / NOT IN)

- IN and NOT IN Operators – Subquery
- Example: List the names of suppliers who supply part ‘p2’.

SELECT SNAME

FROM S

WHERE S# IN (SELECT S# FROM SP
WHERE P# = 'p2')

- IN → Membership operator
 - = ANY or = SOME
- Execution Mechanism:
 - The system first executes the inner SELECT statement.
 - Then, for each row in the outer SELECT, it checks whether the value of S# exists in the set returned by the inner SELECT.
 - If it exists → the row satisfies the condition and is included in the result.

Retrieving Data from More Than One Table – Correlated Subqueries

- Correlated Subquery
- A correlated subquery is a subquery that refers to a column from the outer query.
- It is re-evaluated for each row processed by the outer query.

```
SELECT SNAME  
FROM S  
WHERE 'p2' IN ( SELECT P# FROM SP  
                WHERE SP.S# = S.S# )
```

Retrieving Data from More Than One Table – Comparison Operators (ANY, SOME, ALL)

- Comparison Operators with Subqueries:
- $\theta \in \{ =, \neq, <, \leq, >, \geq \}$

- Possible Forms:

- θ ANY
- θ SOME
- θ ALL

```
1- SELECT S#  
   FROM S  
   WHERE STATUS < ANY ( SELECT DISTINCT STATUS FROM S )
```

```
2- SELECT S#  
   FROM S  
   WHERE STATUS < ( SELECT MAX (STATUS) FROM S )
```



- Example:
- List the supplier numbers (S#) whose status is not the highest.

Retrieving Data from More Than One Table (continued)

```
SELECT SNAME
FROM S
WHERE 0 < ( SELECT COUNT(*)
            FROM SP
            WHERE SP.S# = S.S#
            AND
            SP.P# = 'p2' )
```

Subqueries Using EXISTS and NOT EXISTS

- EXISTS Operator:
- The EXISTS operator returns TRUE if the subquery returns one or more rows.

```
SELECT SNAME
FROM S
WHERE EXISTS (SELECT *
              FROM SP
              WHERE SP.S# = S.S#
                 AND
                 SP.P# = 'p2' )
```

Types of JOIN in SQL

- INNER JOIN
 - Returns only the rows with matching values in both tables.
- LEFT OUTER JOIN
 - Returns all rows from the left table, and the matching rows from the right table. Non-matching rows from the right table are shown as NULL.
- RIGHT OUTER JOIN
 - Returns all rows from the right table, and the matching rows from the left table. Non-matching rows from the left table are shown as NULL.
- FULL OUTER JOIN
 - Returns all rows when there is a match in either left or right table. Non-matching rows in both sides are filled with NULLs.
- NATURAL JOIN
 - Automatically joins tables based on columns with the same names and compatible data types.

INNER JOIN – Example

- List supplier names and the parts they supply.

```
SELECT SNAME  
FROM S INNER JOIN SP  
ON S.S# = SP.S#
```

- Note:
 - Every JOIN (except NATURAL JOIN) must have a condition using ON.

Left Outer Join Example:

Example: List all suppliers and the parts they supply.

```
SELECT S.S#, S.SNAME, SP.P#, SP.PNAME  
FROM S LEFT OUTER JOIN SP  
      ON S.S# = SP.S#
```

End of Chapter

